

Cheat Sheet for Data Analysis #2

Descriptive Statistics

Basic Dataset Inspection

Print numbers in full instead of scientific notation

```
pd.set_option("display.float_format", "{:.2f}".format)
```

Meaning: Without this line pandas prints a large variance as 1.158120e+07; with it the same number reads 11581196.57. Set it once, at the top of the notebook

Load a CSV file into a DataFrame

```
df = pd.read_csv("Diamond.csv")
```

Meaning: Reads data from a CSV file and stores it in a variable called `df`

View the shape of the DataFrame (rows, columns)

```
print(df.shape)
```

Meaning: `.shape` returns a tuple: (number of rows, number of columns); `df.shape[0]` means rows and `df.shape[1]` means columns

List the column names

```
print(df.columns.tolist())
```

Meaning: `.columns` holds the column names and `.tolist()` turns them into an ordinary Python list, which is easier to read and to copy from than the pandas object

Display data types of each column

```
print(df.dtypes)
```

Meaning: `.dtypes` shows the data type (e.g., object, int64, float64) of each column

Check for missing values

```
print(df.isnull().sum())
```

Meaning: Shows the count of missing values for each column - essential to check before analysis

Get concise summary of the DataFrame

```
print(df.info())
```

Meaning: `.info()` shows column names, non-null counts, data types, and memory usage

Show the first few rows of the DataFrame

```
print(df.head(10))
```

Meaning: `.head(n)` displays the first n rows (default is 5) - useful for quick inspection

Selecting Rows and Creating Columns

Select the rows that satisfy a condition

```
expensive = df[df['price'] > 10000]  
print(len(expensive))
```

Meaning: Inside the square brackets goes a condition, and pandas returns a new table containing only the rows for which it is true. `len()` then counts them

Combine two conditions

```
extremes = df[(df['price'] < 1000) | (df['price'] > 15000)]
```

```
print(len(extremes))
```

Meaning: | means “or” and & means “and”. Each condition must be wrapped in its own round brackets, otherwise Python reads the line in the wrong order and reports an error. This is the pattern used to count values outside the outlier bounds

Create a new column

```
df['price_k'] = df['price'] / 1000
```

Meaning: Assigning to a name that does not exist yet adds a column to the table. The right-hand side may be arithmetic on existing columns, and it is computed row by row

Work on a copy rather than on the original table

```
df_scaled = df.copy()
df_scaled['price_k'] = df_scaled['price'] / 1000
```

Meaning: .copy() makes an independent table, so changes to it leave the original untouched. Use it whenever you are about to modify data you may still need in its original form

Descriptive Statistics for Numerical Variables

Sample size (number of observations)

```
n_obs = len(df[['carat', 'price']])
print(f"Sample size: {n_obs}")
```

Meaning: Counts the total number of records in the dataset for numerical analysis

Minimum values

```
print(df[['carat', 'price']].min())
```

Meaning: Shows the smallest value for each numerical variable - helps identify potential outliers and impossible values

Maximum values

```
print(df[['carat', 'price']].max())
```

Meaning: Shows the largest value for each numerical variable - helps identify potential outliers and impossible values

Sample mean (average)

```
print(df[['carat', 'price']].mean())
```

Meaning: Calculates the arithmetic average; represents the “center of mass” of the data but can be influenced by outliers

Median

```
print(df[['carat', 'price']].median())
```

Meaning: The middle value when data is sorted; robust to outliers; better represents “typical” value in skewed distributions

Compare the mean with the median

```
print(df[['carat', 'price']].mean() - df[['carat', 'price']].median())
```

Meaning: A large positive difference means the distribution has a long tail to the right; a value close to zero means the distribution is nearly symmetric

Sample variance

```
print(df[['carat', 'price']].var(ddof=1))
```

Meaning: Measures the average squared deviation from the mean; ddof=1 specifies sample variance (N-1 degrees of freedom), used because the data are a sample rather than the whole population

Sample standard deviation

```
print(df[['carat', 'price']].std(ddof=1))
```

Meaning: Square root of variance; in same units as original data; higher value means data points are more spread out

Quartiles and Quantiles

```
print(df[['carat', 'price']].quantile([0.25, 0.50, 0.75]))
```

Meaning: Returns values at specified percentiles; Q1 (25%), Q2/median (50%), Q3 (75%). Use [0.05, 0.95] to describe the tails

Interquartile range and the outlier rule

```
q1 = df['price'].quantile(0.25)
q3 = df['price'].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df[(df['price'] < lower) | (df['price'] > upper)]
print(iqr, lower, upper, len(outliers))
```

Meaning: $IQR = Q3 - Q1$ is the width of the middle half of the data and ignores the extremes entirely. A value outside $Q1 - 1.5 * IQR$ or $Q3 + 1.5 * IQR$ is conventionally called an outlier. The same rule draws the whiskers of a box plot

Skewness

```
print(df[['carat', 'price']].skew())
```

Meaning: Measures asymmetry of distribution; >0 indicates right-skewed (long tail to right), <0 indicates left-skewed, 0 means symmetric

Kurtosis

```
print(df[['carat', 'price']].kurtosis())
```

Meaning: Measures tail heaviness; pandas returns EXCESS kurtosis, so 0 corresponds to a normal distribution, not 3. Textbooks that report the uncorrected value differ from this one by exactly 3

Sort a column, or a whole table of statistics

```
print(df[['carat', 'price']].skew().sort_values(ascending=False))
```

Meaning: `.sort_values()` orders the result; `ascending=False` puts the largest value first, which is convenient when several columns are being ranked

All of the above in one line

```
print(df[['carat', 'price']].describe().round(2))
```

Meaning: `.describe()` returns count, mean, standard deviation, minimum, the three quartiles and the maximum. `.round(2)` keeps the output readable. Compute the statistics by hand first, then use this as the shortcut

Descriptive Statistics for Categorical Variables

Frequency tables

```
print(df['colour'].value_counts().sort_index())
```

Meaning: `.value_counts()` shows how many observations fall into each category; `.sort_index()` orders the categories themselves instead of ordering them by count, which matters for an ordered scale

Shares instead of counts

```
print((df['colour'].value_counts(normalize=True).sort_index() * 100).round(1))
```

Meaning: `normalize=True` returns proportions instead of counts. Shares are easier to compare than

counts and allow two datasets of different size to be compared directly

Multiple column frequency counts

```
print(df[['colour', 'certification']].value_counts())
```

Meaning: Counts occurrences of unique combinations across multiple categorical columns

Mode (most frequent value)

```
print(df[['colour', 'clarity', 'certification']].mode().iloc[0])
```

Meaning: Identifies the most frequently occurring category; for a categorical column it is the only measure of the centre available. `.mode()` returns every value tied for first place, so check the full list before taking `.iloc[0]`

describe() for categorical columns

```
print(df[['colour', 'clarity', 'certification']].describe())
```

Meaning: For text columns `describe()` reports the number of observations, the number of distinct categories, the most frequent category and its frequency

Turn a numerical column into categories

```
import numpy as np
bins = [0, 50, 250, np.inf]
labels = ["Small", "Medium", "Large"]
df['size_band'] = pd.cut(df['labour'], bins=bins, labels=labels, right=True)
```

Meaning: `pd.cut` assigns every row to an interval. `right=True` closes each interval on the right, so a value of exactly 50 falls into the first band. `np.inf` as the last edge removes the need to know the maximum in advance. The thresholds should come from an external classification, not from the data. The example above uses `Labour.csv` from the seminar tasks; `Diamond.csv` has no `labour` column, so there the same command would be applied to `carat`

Comparing Groups

The same statistic computed separately for each group

```
print(df.groupby('certification')['price'].agg(['mean', 'median', 'count']).round(2))
```

Meaning: `groupby` splits the rows into groups, applies the same statistic to each part and combines the results into one table. It answers questions of the form “does this variable differ across those categories?”

Rank the largest observations and measure concentration

```
top = df.nlargest(30, 'price')
print(top['price'].sum() / df['price'].sum() * 100)
```

Meaning: `.nlargest(n, column)` returns the `n` rows with the largest values. Dividing their sum by the total of the column gives the share those rows account for, which is the direct way to measure concentration

Bivariate Analysis

Covariance matrix

```
print(df[['carat', 'price']].cov())
```

Meaning: Measures how two numerical variables change together; positive = variables increase together, negative = inverse relationship. Its magnitude depends on the units of both variables, so the number itself cannot be compared across pairs

Read one value out of a matrix

```
cov_matrix = df[['carat', 'price']].cov()
```

```
print(cov_matrix.loc['carat', 'price'])
```

Meaning: `.loc[row, column]` picks a single cell by the NAMES of its row and column, which is how one number is extracted from a covariance or correlation matrix

Correlation between two variables

```
print(df['carat'].corr(df['price']))
```

Meaning: Measures strength and direction of a linear relationship (-1 to 1); close to 1 = strong positive linear relationship. It is the covariance divided by both standard deviations, which removes the units

Correlation matrix (all numerical variables)

```
print(df[['carat', 'price']].corr())
```

Meaning: Shows pairwise correlations between all numerical variables; diagonal is always 1; high off-diagonal values indicate strong linear relationships

Contingency table (two categorical variables)

```
print(pd.crosstab(df['colour'], df['certification']))
```

Meaning: Shows joint frequency distribution of two categorical variables; reveals associations between categories

The same table as row percentages

```
print((pd.crosstab(df['colour'], df['certification'], normalize='index') * 100).round(1))
```

Meaning: `normalize='index'` divides each row by its own total. Raw counts cannot be compared across rows of different size; row percentages can. If every row showed the same split, the two variables would be unrelated

Practical Tips

1. **Always check for missing data** before performing statistical analysis
2. **Compare mean and median** - if mean > median, distribution is likely right-skewed
3. **Use median and IQR** for skewed distributions instead of mean and standard deviation
4. **An outlier is not an error.** Check whether the extreme value is impossible before deciding that it is wrong; in some datasets the extreme observations are the ones that matter most
5. **A correlation computed over a mixed population** can be much weaker than the correlation inside each of its parts. Split the sample by a category and look again
6. **A share is easier to understand than a count.** Report “two thirds of output” rather than the absolute figure
7. **Visualize your data** with histograms and box plots to complement numerical statistics

Remember: correlation does not imply causation, and it only measures linear relationships. A coefficient near zero rules out a straight line, not a relationship.